

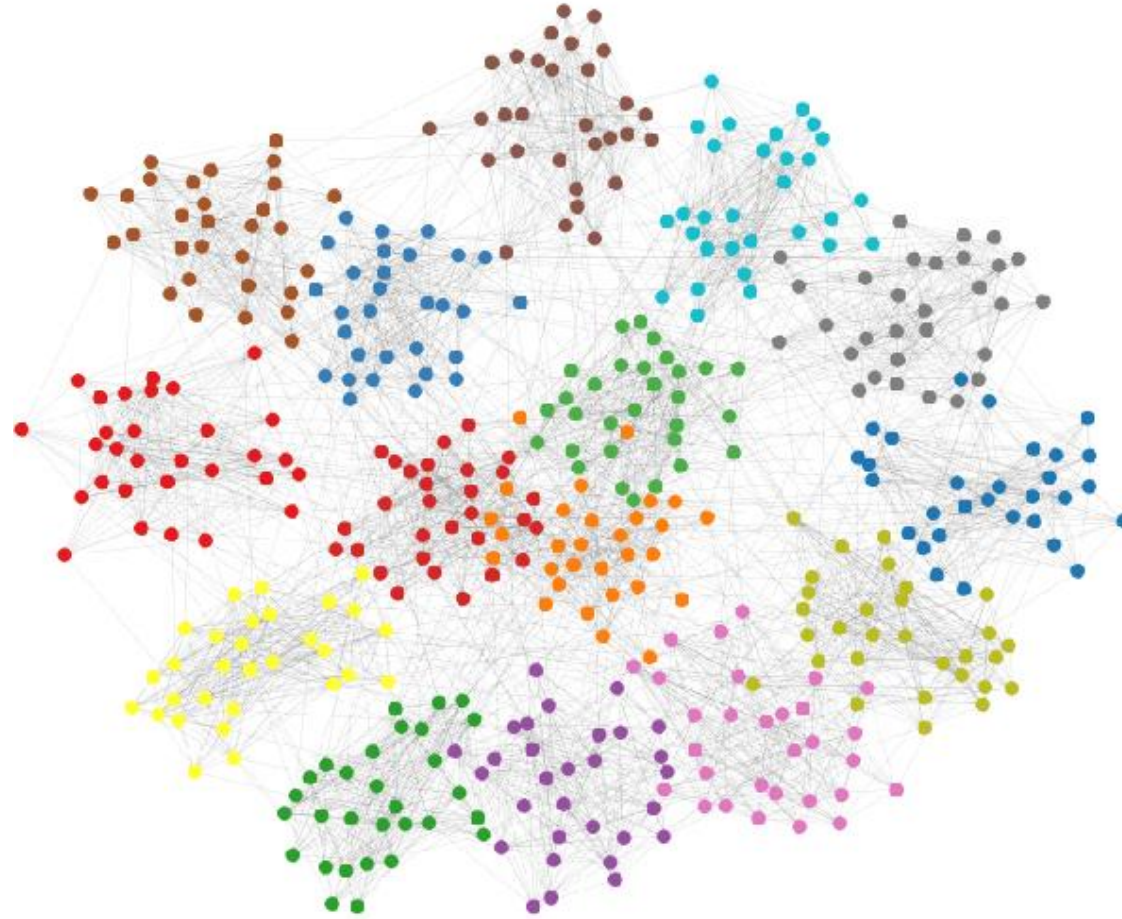
Detecting Communities under Differential Privacy

Hiep H. Nguyen

Abdessamad Imine

Michaël Rusinowitch

Community Detection



Definitions

Symbol	Definition
$G = (V, E_G)$	true graph with $n = V $ and $m = E_G $
$G' = (V, E_{G'})$	neighboring graph of G
$\tilde{G} = (V, E_{\tilde{G}})$	sample noisy output graph
$G_1 = (V_1, E_1)$	supergraph generated by LouvainDP
k	fan-out of the tree in ModDivisive
K	burn-in factor in MCMC-based algorithms
λ	common ratio to distribute the privacy budget
C	a clustering of nodes in G
$Q(G, C)$	modularity of the clustering C on graph G

Motivation

- Community Detection (CD) is an important tool for analyzing graph networks
- Research rarely tries to conduct CD with privacy guarantees

Previous Work

- ϵ -DP k-Means Clustering
 - ϵ -DP k-Means are low dimension and number of clusters k is usually small
 - ϵ -DP k-Means are normalized to $[-1,1]^d$
 - Found that modularity scores were no better than random clustering
- ϵ -DP recommender systems
 - Hard unless ϵ is relaxed up to $\log n$ (Banerjee et al.)

Modularity

- Modularity is defined as

$$Q = \sum_{c=1}^{n_c} \left[\frac{l_c}{m} - \left(\frac{d_c}{2m} \right)^2 \right]$$

- Where n_c is the number of clusters, l_c is the total number of edges joining nodes in community c and d_c is the sum of the degrees of the nodes of a cluster C

ϵ -Differential Privacy

This paper uses the following privacy metric. A mechanism $\mathcal{A}$ is ϵ -Differential if for any two neighboring graphs G_1 and G_2 , and for any output $O \in \text{Range}(\mathcal{A})$

$$\Pr[\mathcal{A}(G_1) \in O] \leq e^\epsilon \Pr[\mathcal{A}(G_2) \in O]$$

Given two graphs, $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are neighbors if $V_1 = V_2$ and $|E_2| = |E_1| + 1$. This is known as edge differential privacy.

Geometric Mechanism

- Discrete variant of Laplace mechanism with integral output range $\mathbb{Z}$ and random noise Δ generated from a two-sided geometric distribution

$$\text{Geom}(\alpha) : \Pr[\Delta = \delta | \alpha] = \frac{1 - \alpha}{1 + \alpha} \alpha^{|\delta|}$$

- Can satisfy ϵ -DP by setting $\alpha = \exp(-\epsilon)$

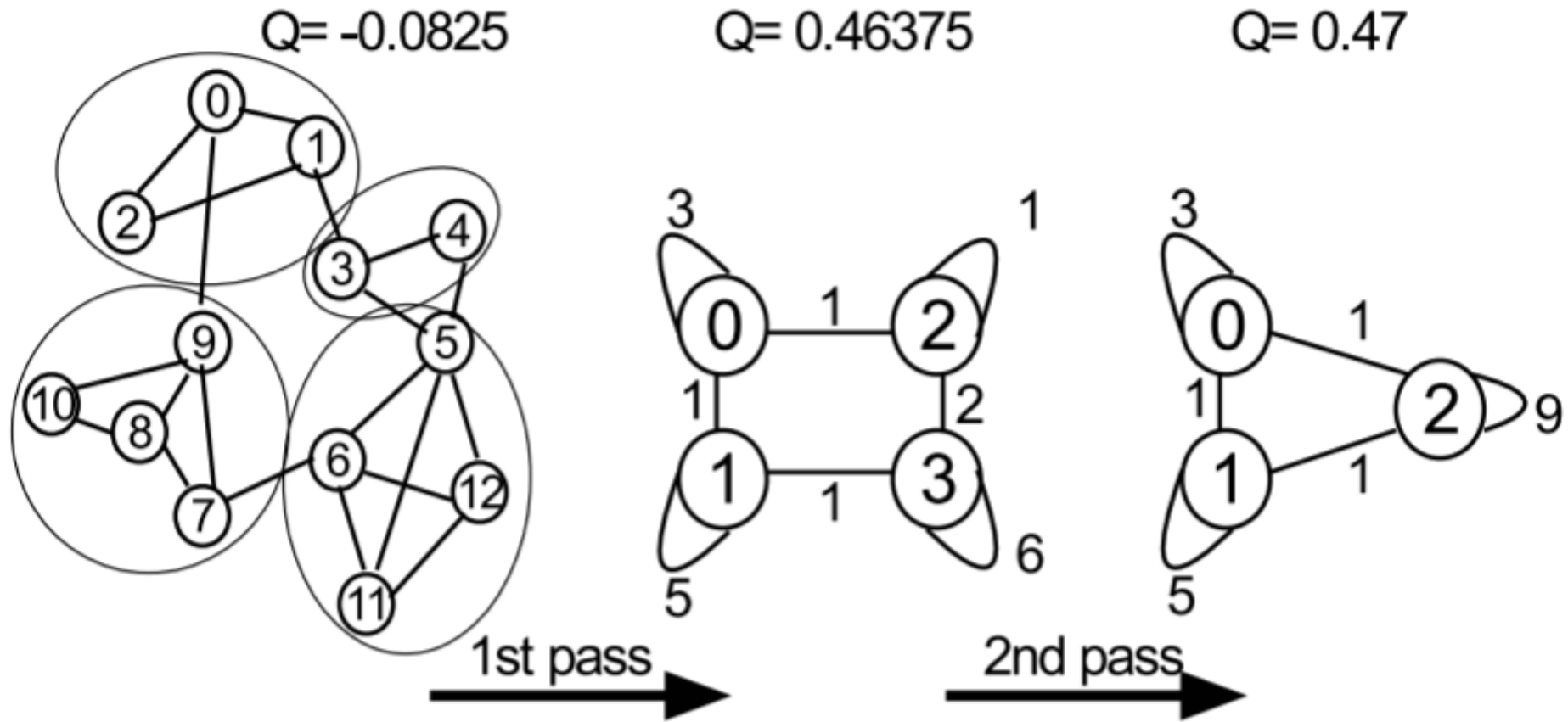
Important elements of Differential Privacy

- **Privacy Budget** : Inverse to sensitivity. A budget associated for each for each query and effectively limits how many operations can be done on the query before it is no longer considered anonymous
- **Noise Distribution** : some mechanism like Laplacian or Geometrical is typically used
- **Global Sensitivity** : A measure of differential privacy
 - $\Delta f = \max_{G_1, G_2} \|f(G_1) - f(G_2)\|_1$

Two Main Algorithms

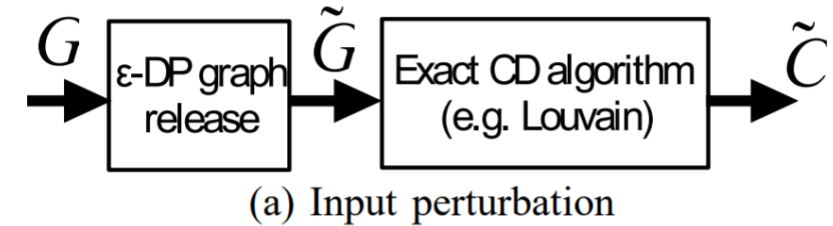
- Louvain Method
 - Input Perturbation
- ModDivisive
 - Algorithm Perturbation

Louvain Method



LouvainDP

- Input Perturbation
 - Build noisy weighted super graph
- Randomly split nodes of graph G into groups of size k as graph G_1
- Compute Edges of G_1
 - Add non-zero superedges between super nodes with modularity $\geq \theta$
 - Add s zero superedges with some weights from a distribution
- Privacy Budget
 - Geometric mechanism ϵ_1
 - Computing non-zero super edges ϵ_2
 - Number of nodes and group size is public
- $O(m)$



Algorithm 1 LouvainDP(G, s)

Input: undirected graph G , group size k , privacy budget ϵ

Output: noisy partition $\tilde{C}$

```

1:  $G_1 \leftarrow \emptyset$ ,  $n_1 = \lfloor \frac{|V|}{k} \rfloor - 1$ ,  $V_1 \leftarrow \{0, 1, \dots, n_1\}$ 
2:  $\epsilon_2 = 0.1$ ,  $\epsilon_1 = \epsilon - \epsilon_2$ ,  $\alpha = \exp(-\epsilon_1)$ 
3: get a random permutation  $V_p$  of  $V$ 
4: compute the mapping  $M : V_p \rightarrow V_1$ 
5: compute superedges of  $G_1$ :  $E_1 = \{e_1(i, j)\}$  where  $i, j \in V_1$ 
6:  $m_1 = |E_1| + \text{Lap}(1/\epsilon_2)$ ,  $m_0 = \frac{n_1(n_1+1)}{2}$ 
7:  $\theta = \lceil \log_\alpha \frac{(1+\alpha)m_1}{m_0 - m_1} \rceil$ 
8:  $s = (m_0 - m_1) \frac{\alpha}{1+\alpha}$ 
9: for  $e_1(i, j)$  in  $E_1$  do
10:    $e_1(i, j) = e_1(i, j) + \text{Geom}(\alpha)$ 
11:   if  $e_1(i, j) \geq \theta$  then
12:     add  $e_1(i, j)$  to  $G_1$ 
13: for  $s$  edges sampled uniformly at random  $e_1(i, j) \notin E_1$  do
14:   draw  $w$  from the distribution  $\text{Pr}[X \leq x] = 1 - \alpha^{x-\theta+1}$ 
15:   if  $w > 0$  then
16:     add edge  $e_1(i, j)$  with weight  $w$  to  $G_1$ 
17: run Louvain method on  $G_1$  to get  $\tilde{C}_1$ 
18: compute  $\tilde{C}$  from  $\tilde{C}_1$  using the mapping  $M$ 
19: return  $\tilde{C}$ 
  
```

LouvainDP Runtime

Proof for runtime $O(m)$

- Line 5 and Lines 9-12 cost $O(m)$
- Processing of s super edges costs $O(m)$

$$s = (m_0 - m_1) \frac{\alpha^\theta}{1 + \alpha} \leq \frac{m_0 - m_1}{1 - \alpha} \frac{(1 + \alpha)m_1}{m_0 - m_1} = m_1$$

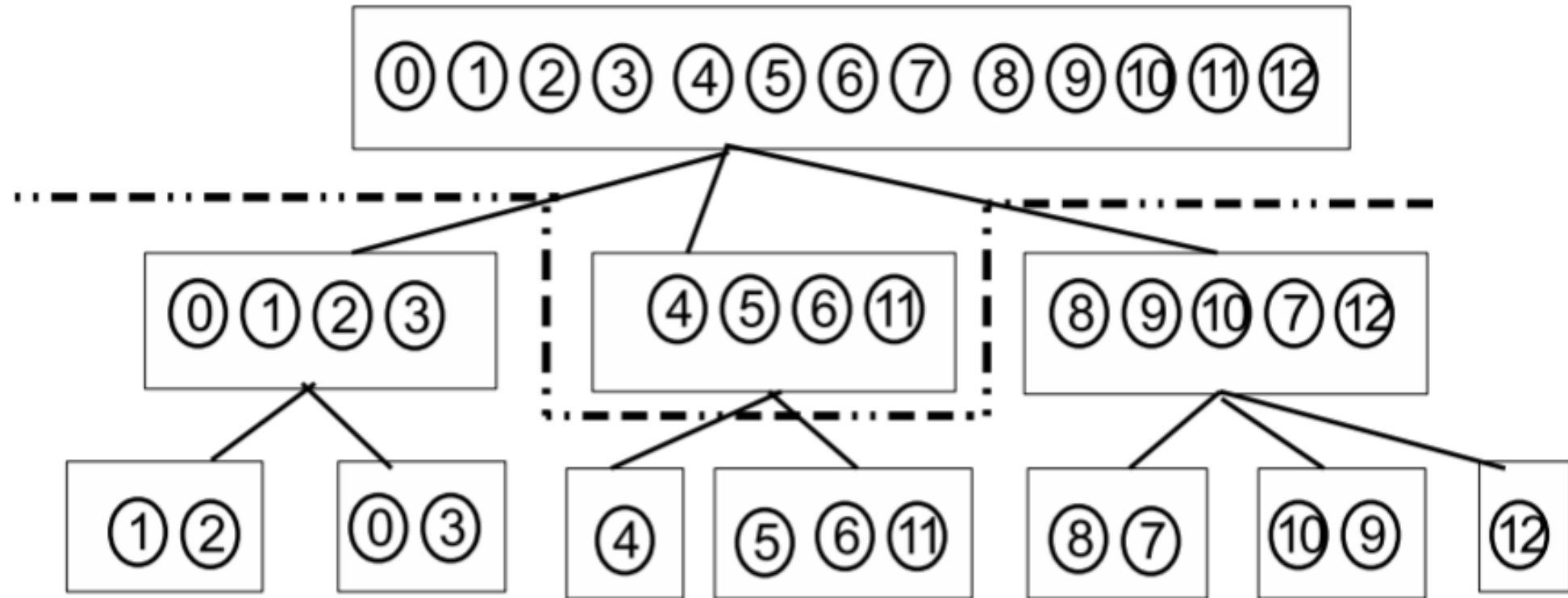
- Louvain Method is empirically linear in m_1
- The number of edges in $G_1 = m_1 \leq 2m$
- The algorithm is therefore bounded by $O(m)$

Proof of Privacy

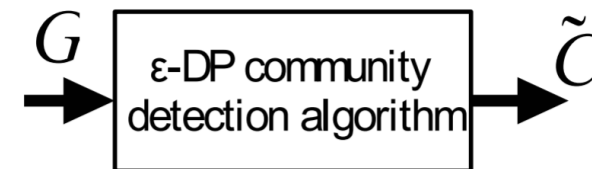
- Note: Getting a random permutation V_p costs no privacy budget
- A small privacy budget $\epsilon_2 = 0.01$ to compute noisy number of non-zero super edges m_1
- Remaining budget ϵ_1 is used for $Geom(\alpha)$
- LouvainDP satisfies ϵ -differential privacy by setting $\epsilon_1 = \epsilon - \epsilon_2$

Theorem 3.3: (Sequential and parallel compositions [22])
Let each A_i provide ϵ_i -differential privacy. A sequence of $A_i(D)$ over the dataset D provides $\sum_{i=1}^n \epsilon_i$ -differential privacy.
Let each A_i provide ϵ_i -differential privacy. Let D_i be arbitrary disjoint subsets of the dataset D . The sequence of $A_i(D_i)$ provides $\max_{i=1}^n \epsilon_i$ -differential privacy. $\square$

ModDivisive



ModDivisive



- Algorithm Perturbation
- Build tree
 - Construct layers of k -ary tree with depth $maxL$ using Markov Chain MonteCarlo
 - Find best cut of final tree
- Privacy Budget
 - k -ary tree uses e_1 budget
 - BestCut takes ϵ_m at each level
- $O(m)$

Algorithm 3 ModDivisive

Input: graph G , group size k , privacy budget ϵ , max level $maxL$, ratio λ , BestCut privacy at each level ϵ_m

Output: noisy partition $\tilde{C}$

```
1: compute the array  $eA[0..maxL-1]$  s.t.  $\sum_i eA[i] = \epsilon_1$ ,  $eA[i] = eA[i+1] * \lambda$  where  $\epsilon_1 = \epsilon - maxL * \epsilon_m$ 
2: initialize the root node with nodeset  $V$ 
3:  $root = NodeSet(G, V, k)$ 
4:  $root.level = 0$ 
5: queue  $Q \leftarrow root$ 
6: while  $Q$  is not empty do
7:    $r \leftarrow Q.dequeue()$ 
8:   if  $r.level < maxL$  then
9:      $r.part = ModMCMC(G, r.S, k, eA[r.level])$ 
10:    for subset  $S_i$  in  $r.part$  do
11:       $P_i = NodeSet(G, S_i, k)$ 
12:       $P_i.level = r.level + 1$ 
13:       $r.child_i \leftarrow P_i$ 
14:       $Q.enqueue(P_i)$ 
15:  $\tilde{C} \leftarrow BestCut(root, \epsilon_m)$ 
16: return  $\tilde{C}$ 
```

ModMCMC

- Global Sensitivity of modularity is bounded by $O(1/m)$
- Until equilibrium is reached
 - Random partition by moving a node u into another group and accept change with some probability
- $O(K * |V| * \max L)$ total

Algorithm 4 ModMCMC

Input: graph G , nodeset $r.S$, group size k , privacy budget ϵ_p

Output: sampled partition $r.part$

- 1: initialize $r.part$ with a random partition P_0 of k groups
 - 2: **for** each step i in the Markov chain **do**
 - 3: pick a neighboring partition P' of P_{i-1} by randomly selecting node $u \in r.S$ and moving u to another group.
 - 4: accept the transition and set $P_i = P'$ with probability $\min(1, \frac{\exp(\frac{\epsilon_p}{2\Delta Q} Q(P', G))}{\exp(\frac{\epsilon_p}{2\Delta Q} Q(P_{i-1}, G)})$
 - 5: // until equilibrium is reached
 - 6: **return** a sampled partition $r.part = P_i$
-

BestCut

- Using a stack with nodes of the tree in reversed level order
 - Pop a node r and find $opt(r)$ and either accept r as a solution or apart of a subtree of the solution based on a Laplace distribution
- Sublinear because the size of tree is always smaller than $|V|$

Algorithm 5 BestCut

Input: undirected graph G , root node $root$, privacy budget at each level ϵ_m

Output: best cut C

```
1: stack  $S \leftarrow \emptyset$ , queue  $Q \leftarrow root$ 
2: while  $Q$  is not empty do
3:    $r \leftarrow Q.dequeue()$ 
4:    $S.push(r)$ 
5:   for child node  $r_i$  in  $r.children$  do
6:      $Q.enqueue(r_i)$ 
7: dictionary  $sol \leftarrow \emptyset$ 
8: while  $S$  is not empty do
9:    $r \leftarrow S.pop()$ ,  $r.mod_n = r.mod + \text{Laplace}(\Delta Q / \epsilon_m)$ 
10:  if  $r$  is a leaf node then
11:     $sol.put(r.id, (val = r.mod_n, self = True))$ 
12:  else
13:     $s_m = \sum_{r_i \in r.children} sol[r_i.id].mod_n$ 
14:    if  $r.mod_n < s_m$  then
15:       $sol.put(r.id, (val = s_m, self = False))$ 
16:    else
17:       $sol.put(r.id, (val = r.mod_n, self = True))$ 
18: list  $C \leftarrow \emptyset$ , queue  $Q \leftarrow root$ 
19: while  $Q$  is not empty do
20:    $r \leftarrow Q.dequeue()$ 
21:   if  $sol[r.id].self == True$  then
22:      $C = C \cup \{r\}$ 
23:   else
24:     for child node  $r_i$  in  $r.children$  do
25:        $Q.enqueue(r_i)$ 
return  $C$ 
```

Results

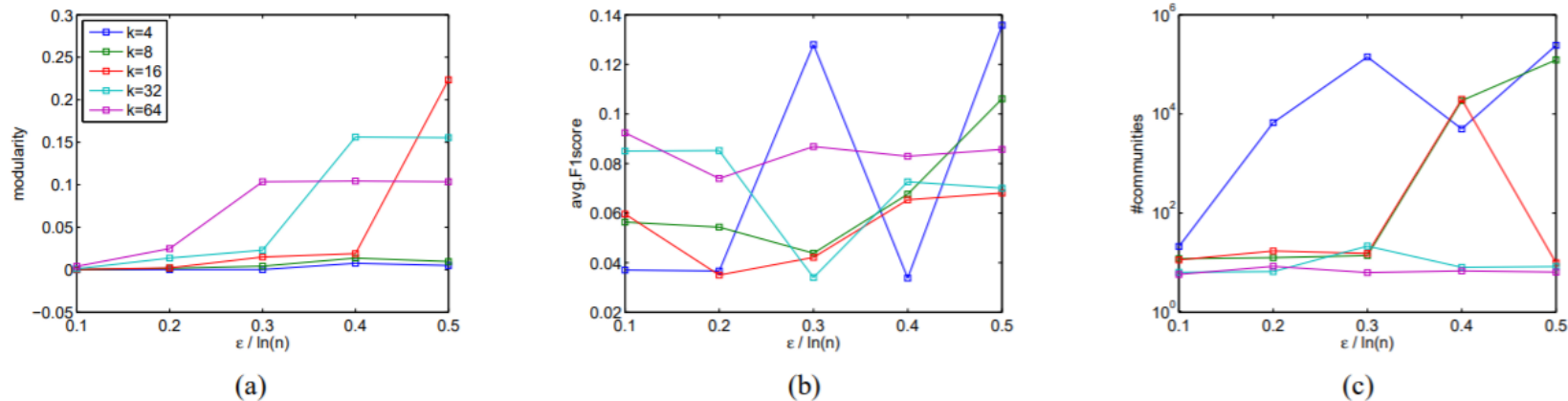


Fig. 5: LouvainDP on youtube ($0.5 \ln n = 7.0$)

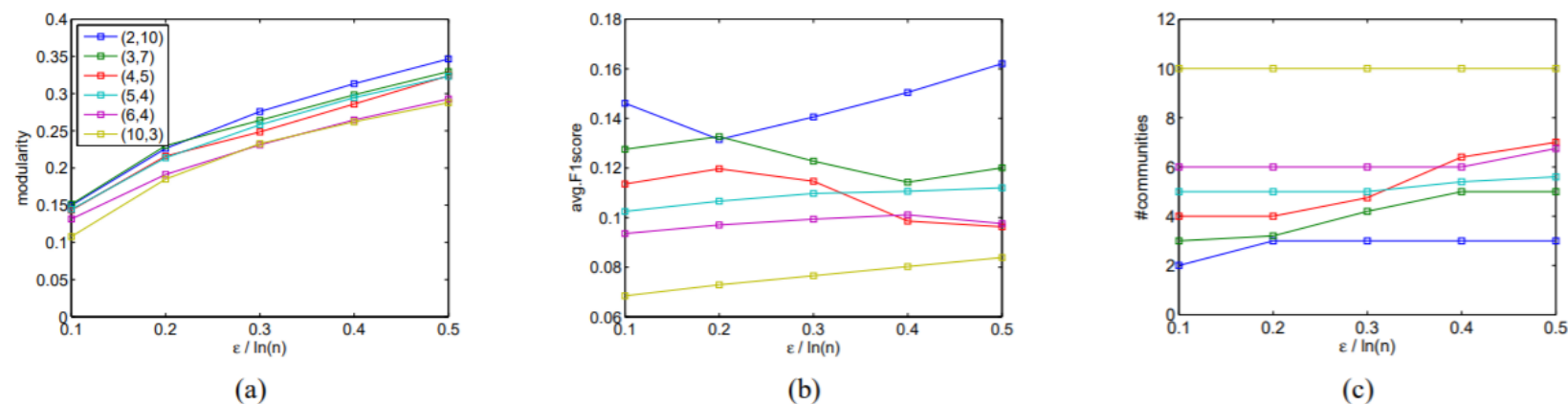
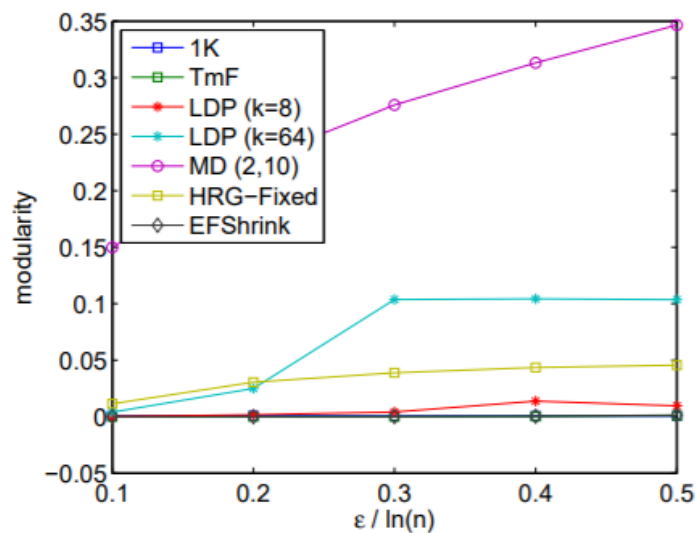
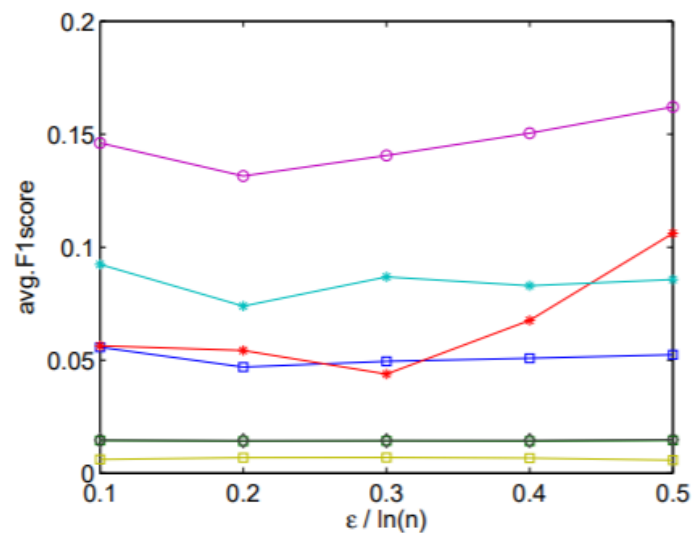


Fig. 6: ModDivisive on youtube with $\lambda = 2.0$, $K = 50$ ($0.5 \ln n = 7.0$)

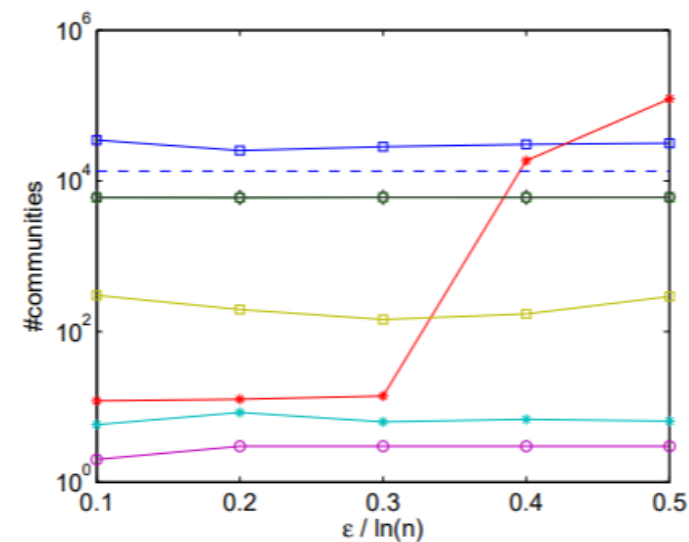
Results



(a)



(b)



(c)

Fig. 13: Quality metrics and the number of communities (youtube) ($0.5 \ln n = 7.0$)

Important Challenges

- High dimensionality of data
- High number of communities
- High sensitivity $O(n)$
- Generalized graph structure
- Modularity is non-monotonic function

Limitations

- Limited to very large datasets
- Once a dataset is differentially private, it is limited to a single query
 - Must regenerate DP data for each query
- Research limited to edge differential privacy
 - group size (k) and number of nodes (n) and super nodes (n_1) are public
- Low privacy budget
 - $0.5 \ln n$ vs $\ln n$